Ad Soyad: _____	Tarih: _____
Proje Ödevi – KUBYS – *Kütüphane Yönetim Sistemi*	

Proje Hakkında

Bu projede bir **Kütüphane Yönetim Sistemi (KUBYS)** geliştireceksiniz. Sistem; kitap kataloğu, üye yönetimi ve ödünç alma/iade takibi işlevlerini içerecektir. Proje dönem boyunca 4 fazda geliştirilecek ve her faz bir öncekinin üzerine inşa edilecektir.

- **Faz 1 (Hafta 1–3):** STL kapsayıcıları ve modern C++ ile veri katmanı
- **Faz 2 (Hafta 4–8):** Qt6 ile grafiksel kullanıcı arayüzü
- **Faz 3 (Hafta 9–11):** Çok kanallı programlama ile performans iyileştirmesi
- **Faz 4 (Hafta 12–14):** Ağ programlama ile istemci/sunucu mimarisi

Süre: 14 hafta (dönem boyunca) **Başlangıç kodu:** `out/Projects/KUBYS/`

Teknoloji Gereksinimleri

Bu proje **C++20** standardı ve **Qt6** çerçevesi kullanılarak geliştirilmelidir. Aşağıdaki araçların sisteminizde kurulu olması gerekir:

- **Derleyici:** g++ veya clang++ (C++20 desteği ile)
- **Qt6:** Qt 6.5+ (Widgets, PrintSupport, Charts modülleri)
- **CMake:** 3.20 veya üzeri

Faz 1 derleme:

```
g++ -std=c++20 -o kubys src/main.cpp
```

Faz 2+ derleme (Qt ile):

```
mkdir build && cd build
cmake .. -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x
cmake --build .
```

1 Alan Modeli

Sisteminiz aşağıdaki temel varlıkları içerecektir:

Varlık	Anahtar	Alanlar
`Kitap`	`isbn`	başlık, yazar, yayın yılı, kategori, kopya sayısı
`Uye`	`uye_no`	isim, soyisim, telefon, kayıt tarihi
`OduncKaydi`	`kayit_id`	üye no, isbn, ödünç tarihi, iade tarihi, durum

Durum Enum'ları

```cpp
enum class OduncDurum {
    Oduncte,        // Kitap hala ödünç alınmış durumda
    IadeEdildi,     // Kitap iade edildi
    Gecikmis        // İade süresi geçmiş
};
```

Örnek Struct Tanımı

```cpp
struct Kitap {
    std::string isbn;
    std::string baslik;
    std::string yazar;
    int yayin_yili{};
    std::string kategori;
    int kopya_sayisi{};
};
```

Varlık İlişkileri

- Bir `Uye` birden fazla `OduncKaydi`'na sahip olabilir (1:N)
- Bir `Kitap` birden fazla `OduncKaydi`'nda yer alabilir (1:N)
- `OduncKaydi`, `Uye` ve `Kitap` arasında köprü görevi görür

2 Faz 1: STL ve Veri Katmanı (Hafta 1–3)

Faz 1 Hedefi

Bu fazda projenin veri katmanını oluşturacaksınız. Tüm veri yapıları STL kapsayıcıları ile yönetilecek, dosya okuma/yazma işlemleri ikili (binary) formatta gerçekleştirilecektir. **Bu fazda henüz Qt kullanılmayacaktır;** yalnızca C++20 standart kütüphanesi kullanılacaktır.

Görev 1.1 – Varlık Struct'ları

`include/varliklar.h` dosyasında aşağıdaki struct'ları tanımlayın:

- `Kitap`, `Uye`, `OduncKaydi` struct'ları
- `OduncDurum` enum class'ı
- Her struct için `operator«` (akış çıkış operatörü)
- En az bir alanda `std::optional` kullanımı (örneğin iade tarihi)

Görev 1.2 – Depo Şablon Sınıfı

`include/depo.h` dosyasındaki `Depo<K,V>` şablon sınıfını tamamlayın:

- `ekle(anahtar, deger)`: Yeni kayıt ekler
- `bul(anahtar) -> std::optional<V>`: Anahtar ile arama yapar
- `sil(anahtar) -> bool`: Kaydı siler
- `tumunu_al() -> const std::map<K,V>&`: Tüm kayıtları döner
- `filtrele(kosul) -> std::vector<V>`: Lambda ile filtreleme

Görev 1.3 – STL Algoritmaları

Aşağıdaki sorguları STL algoritmaları ve/veya C++20 ranges kullanarak yazın:

- Belirli bir kategorideki tüm kitapları listeleme
- Gecikmiş iade kayıtlarını bulma
- Kitapları yayın yılına göre sıralama
- Bir üyenin toplam ödünç aldığı kitap sayısını hesaplama
- En çok ödünç alınan kitabı bulma

Görev 1.4 – Dosya İşlemleri

Verileri ikili dosyada saklayın ve okuyun:

- `dosyaya_kaydet(dosya_yolu, depo)`: Depo içeriğini dosyaya yazar
- `dosyadan_oku(dosya_yolu) -> Depo`: Dosyadan veri okur
- Program kapanırken otomatik kaydetme, açılırken otomatik yükleme

İpucu: Ranges Pipeline

C++20 ranges ile sorgu yazma örneği:

```
auto geciken = kayitlar
    | std::views::filter([](const auto& k) {
          return k.durum == OduncDurum::Gecikmis;
      })
    | std::views::transform([](const auto& k) {
          return k.isbn;
      });
```

3 Faz 2: Qt6 ile Grafiksel Arayüz (Hafta 4–8)

Faz 2 Hedefi

Bu fazda Faz 1'de oluşturduğunuz veri katmanının üzerine Qt6 ile bir masaüstü arayüzü inşa edeceksiniz. **Qt6 Widgets** modülü kullanılacak ve **CMake** ile derlenecektir.

Görev 2.1 – Ana Pencere

`QMainWindow` tabanlı ana pencere oluşturun:

- Menü çubuğu: Dosya (Kaydet, Yükle, Çıkış), Düzen (Kitap Ekle, Üye Ekle), Yardım
- Araç çubuğu: Sık kullanılan işlemler için butonlar
- Sekmeli görünüm (`QTabWidget`): Kitaplar, Üyeler, Ödünç Kayıtları
- Durum çubuğu: Kayıt sayıları ve son işlem bilgisi

Görev 2.2 – Veri Giriş Formları

Her varlık için `QDialog` tabanlı giriş formu oluşturun:

- Kitap ekleme/düzenleme dialogu (ISBN, başlık, yazar, yıl, kategori)
- Üye kayıt dialogu (numara, isim, soyisim, telefon)
- Ödünç verme dialogu (üye seçimi `QComboBox`, kitap seçimi, tarih `QDateEdit`)
- Form doğrulama: Boş alan kontrolü, ISBN formatı kontrolü

Görev 2.3 – Model/View Tabloları

Her varlık için `QAbstractTableModel` alt sınıfı oluşturun:

- `KitapModel`: Kitap listesini tablo olarak gösterir
- `UyeModel`: Üye listesini tablo olarak gösterir
- `OduncModel`: Ödünç kayıtlarını gösterir (durum renklendirmeli)
- `QSortFilterProxyModel` ile arama ve filtreleme desteği

Görev 2.4 – Kalıcılık ve Raporlama

- Pencere boyutu ve konumunu `QSettings` ile kaydetme/yükleme
- JSON formatında dışa aktarım (`QJsonDocument`)
- Basit istatistik grafiği (`QChart`): Kategoriye göre kitap dağılımı

İpucu: Sinyal/Slot Bağlantısı

Qt6'da yeni bağlantı söz dizimini kullanın (derleme zamanı kontrolü):

```
connect(ekleButon, &QPushButton::clicked,
        this, &MainWindow::kitapEkle);
```

4 Faz 3: Çok Kanallı Programlama (Hafta 9–11)

Faz 3 Hedefi

Bu fazda uygulamanızı çok kanallı hale getirerek uzun süren işlemlerin arayüzü dondurmamasını sağlayacaksınız. `std::thread`, `std::mutex` ve Qt'nin `QThread` mekanizması kullanılacaktır.

Görev 3.1 – Arka Plan İşlemleri

Aşağıdaki işlemleri arka plan izleklerinde gerçekleştirin:

- Büyük katalog dosyasını arka planda yükleme (ilerleme çubuğu ile)
- Toplu kitap aktarımı (CSV dosyasından)
- Gecikme raporu oluşturma (tüm üyeleri tarayarak)

Görev 3.2 – Senkronizasyon

- Paylaşılan veri yapılarını `std::mutex` / `std::lock_guard` ile koruma
- İlerleme bildirimi: İşçi izlekten GUI'ye sinyal gönderme
- İptal mekanizması: Kullanıcı uzun işlemi iptal edebilmeli

İpucu: QThread ile İlerleme Bildirimi

```
class KatalogYukleyici : public QObject {
    Q_OBJECT
signals:
    void ilerlemeGuncellendi(int yuzde);
    void tamamlandi();
    void hataOlustu(const QString& mesaj);
};
```

5 Faz 4: Ağ Programlama (Hafta 12–14)

Faz 4 Hedefi

Bu fazda uygulamanızı istemci/sunucu mimarisine dönüştüreceksiniz. Merkezi bir sunucu tüm verileri yönetecek, kütüphane terminalleri ise birer istemci olarak bağlanacaktır. **Qt Network** modülü ve **TCP soketleri** kullanılacaktır.

Görev 4.1 – TCP Sunucu

`QTcpServer` ile merkezi sunucu oluşturun:

- Birden fazla istemciyi eş zamanlı kabul etme
- JSON tabanlı mesaj protokolü (istek/yanıt formatı)
- Kitap ödünç verme/iade işlemlerinde çakışma kontrolü

Görev 4.2 – TCP İstemci

`QTcpSocket` ile istemci uygulaması oluşturun:

- Sunucuya bağlanma, kimlik doğrulama
- Uzaktan kitap arama, ödünç alma, iade etme
- Bağlantı kopması durumunda yeniden bağlanma denemesi

İpucu: JSON Mesaj Protokolü

```
// Istek formati
{
    "tip": "odunc_al",
    "uye_no": 1001,
    "isbn": "978-605-1234-56-7"
}

// Yanit formati
{
    "durum": "basarili",
    "mesaj": "Kitap odunc verildi",
    "kayit_id": 42
}
```

6 Değerlendirme Kriterleri

Faz	Puan	Değerlendirme Alanları
Faz 1: STL ve Veri Katmanı	25	Struct tasarımı, Depo şablonu, STL algoritmaları, dosya I/O, `optional`/`variant` kullanımı
Faz 2: Qt GUI	35	Ana pencere, formlar, Model/View, sinyal/slot, QSettings, QChart, kod kalitesi
Faz 3: Çok Kanallı	20	Thread kullanımı, mutex/lock_guard, ilerleme raporlama, iptal mekanizması
Faz 4: Ağ Programlama	20	TCP sunucu/istemci, JSON protokol, çoklu istemci desteği, hata yönetimi
Toplam	**100**	

7 Teslim Edilecekler

Her faz sonunda aşağıdaki dosyaları teslim edin:

Faz 1 Teslimi (Hafta 3 sonu)

- `include/varliklar.h` – Varlık struct'ları ve enum'lar
- `include/depo.h` – Depo şablon sınıfı
- `src/main.cpp` – Test ve demonstrasyon kodu
- `CMakeLists.txt` – Derleme yapılandırması

Faz 2 Teslimi (Hafta 8 sonu)

- Faz 1 dosyaları + Qt GUI kaynak dosyaları
- `src/main_window.h/cpp` – Ana pencere
- `src/kitap_model.h/cpp` – Kitap tablo modeli
- Güncellenmiş `CMakeLists.txt` (Qt6 bağımlılıkları ile)

Faz 3 Teslimi (Hafta 11 sonu)

- Faz 2 dosyaları + thread sınıfları
- En az bir arka plan görevi uygulaması

Faz 4 Teslimi (Hafta 14 – Final)

- Tüm kaynak kodlar
- Sunucu ve istemci uygulamaları
- Kısa kullanım kılavuzu (README.md)
- Proje sunumu (5–10 dakika)